Database Design — Companion Lesson

database-design-companion-lesson.txt

```
================================================================================
 DATABASE DESIGN - COMPANION LESSON
 A bridge between the primer (paper design) and a real, touchable database
================================================================================

 Read the primer first (database-design-primer.txt). This file does not
 replace it. It stands next to it and does three jobs the primer, on purpose,
 does not do:

   1. It shows you how ALL your learning materials fit together, so you never
      feel lost about "which file am I supposed to be in right now."
   2. It makes the abstract ideas concrete: what a database, a table, a row,
      a column, and a query actually ARE as real things - still without asking
      you to install anything.
   3. It clears up the handful of confusions that trip up almost every
      beginner, and for each one it points at exactly where it lives in the
      primer and in the samples PDF, and tells you how the student-panel
      project does it (which is sometimes different - and that is fine).

 Everything here is still READING. You will not touch a computer in this file.
 The touching-a-computer work has its own file: database-design-practice-lesson.txt.
 Do that one AFTER this one.

 This is the FULLER edition of the companion: the same six Parts and the ten
 core confusions as before - plus four more that trip up newcomers nearly as
 often - now with far more drawn-out pictures. Wherever an
 idea can be shown as a real little table with sample rows, or as a diagram you
 can read aloud, it is drawn here rather than only described. Because this is a
 monospace (fixed-width) text file, every table and diagram is hand-drawn so the
 columns line up on screen; read it in a monospace font and the boxes stay square.

 Style note: this file uses the same four signposts as the primer, so they
 should already feel familiar:

   >> CONTEMPLATE   stop and think before reading on. No rush.
   TRY IT          a tiny thing to do on paper.
   KEY IDEA        the one sentence to remember.
   >> PITFALL       a common mistake, named so you can dodge it.

 Take your time. There is no prize for finishing fast. If a part does not
 click, mark it and bring it to me - that is exactly what these files are for.

--------------------------------------------------------------------------------
 PART 1 - THE BIG PICTURE: HOW YOUR MATERIALS FIT TOGETHER
--------------------------------------------------------------------------------

 Before any detail, the map. You now have several documents, written at
 different times and by different helpers. They are not competing. They are
 layers around one idea: how to design a database well. Here is the whole set
 and the single job each one has.

   (1) database-design-primer.txt   <- MY main course. General ER design,
       from scratch, on paper. Parts 0-10 plus a glossary. This is the trunk
       of the tree. If two files ever seem to disagree, we settle it here or
       by thinking it through together - not by picking whichever file we read
       last.
```

(2) database-design-companion-lesson.txt   <- THIS file. The bridge. It
    makes the primer concrete and clears up confusions. Read it second.

(3) database-design-practice-lesson.txt   <- hands-on. You open a free,
    in-browser database and actually build small tables. Read it third,
    and do it with the sandbox open.

(4) database-design-lessons-1....docx   <- a lessons notebook written by
    another AI. Prose lessons (its "Lesson 1" ... "Lesson 19"), plus ten
    practice questions with answers at the end. It covers the SAME ground
    as the primer, with the same examples, in a different voice.

(5) Database Design - Samples ... .PDF   <- the picture-book companion to
    (4). Every example from the notebook, drawn as real tables (with sample
    rows) and as crow's-foot diagrams. This is the best "show me what it
    actually looks like" reference you have.

(6) the student-panel project itself   <- the FINISHED, real design we are
    working toward understanding and re-deriving. We look at it, we do NOT
    change it. You build your own version in student-panel-I.

Here is the same six, drawn as layers around the one idea - method at the top,
the real thing we measure against at the bottom:

```
               THE ONE IDEA: "design a database well, on paper first"
                                    |
          THE METHOD (how to decide anything):
               (1) database-design-primer.txt   <- the trunk
                        /                       \
     SHOWN in prose + pictures            TURNED INTO understanding + skill
     (4) lessons .docx                    (2) THIS companion .txt
     (5) Samples PDF                      (3) practice lesson .txt
                        \                       /
                    MEASURED AGAINST (read-only):
               (6) the student-panel project - the real design
```

KEY IDEA: the primer teaches the method, the notebook+PDF show the same method
in pictures and prose, the companion+practice turn it into understanding and
skill, and student-panel is the real thing we measure our understanding against.

A QUICK "WHICH FILE DO I OPEN?" LOOKUP. Stuck, and not sure where to look?
Match the SHAPE of your question to a file:

When your question sounds like...	reach for...
"HOW do I decide X?"	the primer (the method)
"What does X LOOK like, with rows?"	the samples PDF (the pictures)
"What ARE these things, really?"	this companion, Part 2
"How do I READ this little diagram?"	this companion, Part 3
"Which beginner trap am I in?"	this companion, Part 4
"Do my materials actually agree?"	this companion Part 5, then database-design-AI-pieces-review.txt
"Let me actually BUILD one and see."	the practice lesson (sandbox open)
"What does the finished design do?"	the student-panel project (look only)

>> CONTEMPLATE: which of these do you reach for when you are stuck on "HOW do I
decide X?" (the primer) versus "what does X LOOK like?" (the samples PDF)?
Knowing which tool answers which kind of question saves hours.

A reading order that works well, drawn as a path so you can see the

back-and-forth is on purpose, not you going in circles:

```
  [primer 0-4] --> [companion 1-4] --> [practice lesson] --> [primer 5-10]
        ^                                                          |
        |                                                          v
  (settle disputes here) <----- [re-derive student-panel] <-- [samples PDF]
```

You do not have to follow it exactly; it is a suggestion, and you are allowed
to change it. Tell me if you want to.

PART 2 - FROM PAPER TO A REAL DATABASE: WHAT THESE THINGS ACTUALLY ARE

The primer talks about entities, attributes, and relationships. Those are
PAPER ideas - they live in your head and in your pencil drawing. But a real
database is a real thing on a real computer. This part connects the two, so
the words stop being abstract. We will still only READ here; the practice
lesson is where you make these with your own hands.

Start from the biggest thing and go smaller.

A DATABASE is a labelled box that holds tables. One project, one box. The
student-panel project has exactly one database inside it. Think "a filing
cabinet."

A TABLE is a single grid inside that box: columns across the top, rows going
down. One table holds one KIND of thing - a "students" table, a "parameters"
table. On paper you called this an ENTITY TYPE. Think "one sheet in the filing
cabinet, with a printed header row."

A COLUMN is one labelled slot that every row fills in the same way: "name",
"email", "joined_on". On paper you called this an ATTRIBUTE. Think "one
heading on the sheet."

A ROW is one actual thing: one real student, one real parameter. On paper you
called this an ENTITY INSTANCE. Think "one line filled in on the sheet."

Here is the whole vocabulary on one real grid:

a DATABASE is one labelled box that holds TABLES. Inside it, the table
"student" (paper word: ENTITY TYPE) is a grid of columns and rows:

```
+------------+-------------+------------+
| student_id | full_name   | joined_on  |  <- the COLUMNS (attributes)
+------------+-------------+------------+
|     1      | Sara Ahmadi | 2026-09-01 |  <- one ROW = one instance
|     2      | Omid Karimi | 2026-09-03 |     (one real student)
|     3      | Lena Hassan | 2026-09-05 |
+------------+-------------+------------+
```

- the whole grid is the TABLE (entity type)
- each line of data is a ROW (entity instance)
- each heading is a COLUMN (attribute)
- one cell (a column within one row) holds exactly one VALUE

KEY IDEA: table = the kind of thing (entity type). row = one of them
(instance). column = one fact you keep about each (attribute). If you only
remember one mapping from paper to database, remember this one.

A SCHEMA is the blueprint: the list of tables, their columns, and the rules

(which column is the key, which column points at another table). The schema is the DESIGN - the exact thing the primer is teaching you to produce. The rows are the DATA that later flows into that design. Design the schema well and the data behaves; design it badly and no amount of careful typing saves you.

A QUERY is a question you ask the database in a language called SQL. You will meet four verbs, and only four, for a long time:

```
    CREATE TABLE ...   -> "make a new sheet with these headings" (build design)
    INSERT ...         -> "add one filled-in line"               (add a row)
    SELECT ...         -> "show me the lines that match this"    (ask a question)
    UPDATE / DELETE ...-> "change or remove lines"               (edit data)
```

You do not need to memorise SQL now. You need to see that a QUERY is just you talking to the grid, and that CREATE TABLE is literally your paper design becoming real. In the practice lesson you will type each of these once and watch it happen.

ONE SMALL THING, SHOWN IN ALL THREE FORMS (the core teaching of this Part)

Here is a single idea - "we keep students, each with a name and a join date" - shown as (a) the paper attribute list, (b) the grid it becomes, and (c) the CREATE TABLE that builds it. Same idea, three costumes.

TRIPLE 1 - a plain table (no pointers yet)

  (a) ON PAPER (primer style):

```
    student = (student_id [key], full_name, joined_on)
```

  (b) AS A GRID (samples-PDF style):

```
    +------------+-------------+------------+
    | student_id | full_name   | joined_on  |   <- columns (attributes)
    +------------+-------------+------------+
    |     1      | Sara Ahmadi | 2026-09-01 |   <- a row (one instance)
    |     2      | Omid Karimi | 2026-09-03 |   <- another row
    +------------+-------------+------------+
```

  (c) AS SQL (practice-lesson style):

```
    CREATE TABLE student (
      student_id  INTEGER PRIMARY KEY,
      full_name   TEXT NOT NULL,
      joined_on   DATE
    );
```

Read (a), (b), (c) top to bottom and notice they are the SAME three columns every time. The primer draws (a). The samples PDF draws (b). The practice lesson has you type (c) and then watch the rows from (b) appear. Nothing new is added as you move across - only the costume changes.

The next triples build a tiny LIBRARY whose rows all connect. A member table has the same shape as the student table above (an id, a name, a join date), so we will not redraw it; take members 1 Sara Ahmadi, 2 Omid Karimi, 3 Lena Hassan - the very same three people. Now the new shapes: a book, its physical copies, and the loans that tie members to copies. Keep these ids in mind: members 1-3, books 100-101, copies 500-502, loans 1-3.

TRIPLE 2 - a second plain table, with a natural key kept UNIQUE

  (a) ON PAPER:

```
    book = (book_id [key], title, author, isbn [unique])
```

  (b) AS A GRID:

```
+---------+---------------+-------------+-------------+
| book_id | title         | author      | isbn        |
+---------+---------------+-------------+-------------+
|   100   | The Blue Door | R. Mehta    | 978-0-11-1  |
|   101   | Quiet Rivers  | A. Lindgren | 978-0-22-2  |
+---------+---------------+-------------+-------------+
```

(c) AS SQL:

```
CREATE TABLE book (
  book_id  INTEGER PRIMARY KEY,
  title    TEXT NOT NULL,
  author   TEXT,
  isbn     TEXT UNIQUE
);
```

New thing shown: isbn is a NATURAL key (a real-world identifier). We still make book_id the primary key (a surrogate), and keep isbn UNIQUE so two rows can never claim the same ISBN. That "surrogate PK + unique natural value" pattern is confusion number 3, drawn in full in Part 4.

TRIPLE 3 - a table that POINTS at another (a foreign key)

(a) ON PAPER:

```
copy = (copy_id [key], book_id [FK -> book], shelf_code, acquired_on)
```

(b) AS A GRID (watch book_id - every value is a real book_id from Triple 2):

```
+---------+---------+------------+-------------+
| copy_id | book_id | shelf_code | acquired_on |
+---------+---------+------------+-------------+
|   500   |   100   | A-12       | 2026-01-10  |
|   501   |   100   | A-13       | 2026-01-10  |
|   502   |   101   | B-04       | 2026-02-02  |
+---------+---------+------------+-------------+
```

(c) AS SQL:

```
CREATE TABLE copy (
  copy_id      INTEGER PRIMARY KEY,
  book_id      INTEGER NOT NULL REFERENCES book(book_id),
  shelf_code   TEXT,
  acquired_on  DATE
);
```

New thing shown: book_id in COPY is a FOREIGN KEY. It is not new data about the copy; it is a POINTER at a row in BOOK. Copies 500 and 501 both point at book 100 ("The Blue Door"), so that title has two physical copies. The word REFERENCES in (c) is the database promising to refuse any copy whose book_id is not a real book. This is the one-to-many of Part 4, confusion 4, made of rows.

HOW A POINTER LOOKS ACROSS TWO GRIDS. Read copy.book_id, then find that id up in book:

```
copy 500 -> book_id 100 -> "The Blue Door"
copy 501 -> book_id 100 -> "The Blue Door"   (same title, a second copy)
copy 502 -> book_id 101 -> "Quiet Rivers"
```

The pointer lives on COPY (the many side) and aims at BOOK (the one side).

TRIPLE 4 - the bridge: a table with TWO pointers and a meaningful NULL

(a) ON PAPER:

```
loan = (loan_id [key], member_id [FK -> member], copy_id [FK -> copy],
        borrowed_on, returned_on [NULL = still out])
```

```
  (b) AS A GRID (returned_on is empty on purpose for a loan still out):
      +---------+-----------+---------+-------------+-------------+
      | loan_id | member_id | copy_id | borrowed_on | returned_on |
      +---------+-----------+---------+-------------+-------------+
      |    1    |     1     |   500   | 2026-09-10  |   (NULL)    |
      |    2    |     1     |   502   | 2026-09-02  | 2026-09-09  |
      |    3    |     2     |   501   | 2026-09-13  |   (NULL)    |
      +---------+-----------+---------+-------------+-------------+

  (c) AS SQL:
      CREATE TABLE loan (
        loan_id      INTEGER PRIMARY KEY,
        member_id    INTEGER NOT NULL REFERENCES member(member_id),
        copy_id      INTEGER NOT NULL REFERENCES copy(copy_id),
        borrowed_on  DATE NOT NULL,
        returned_on  DATE               -- NULL until the copy comes back
      );

Two new things shown at once:
  - LOAN carries TWO foreign keys (member_id and copy_id). It sits at the
    "many" end of two one-to-many links, which is what makes it the bridge
    between members and copies. (That is confusion 8, "the one move.")
  - returned_on is NULL for loans 1 and 3. That blank is not missing data by
    accident; it MEANS "not returned yet - still out." Loan 2 has a date, so
    that copy is back. (That is confusion 5, "what NULL means.")
Sara (member 1) appears on two loans; one is still out, one is returned. The
rows connect to Triples 2-3: copy 500 and 502 are real copies, of real books.

TRIPLE 5 - a table that points at ITSELF (a tree)

  (a) ON PAPER:
      student_group = (group_id [key], name, parent_id [FK -> student_group,
                       NULL at the root])

  (b) AS A GRID (parent_id points back into THIS same table; NULL = the root):
      +----------+----------------------+-----------+
      | group_id | name                 | parent_id |
      +----------+----------------------+-----------+
      |    1     | All Students         |  (NULL)   |   <- the root
      |    2     | Grade 10             |     1     |   -> child of 1
      |    3     | Grade 11             |     1     |   -> child of 1
      |    4     | Grade 10 - Section A |     2     |   -> child of 2
      |    5     | Grade 10 - Section B |     2     |   -> child of 2
      +----------+----------------------+-----------+

  (c) AS SQL:
      CREATE TABLE student_group (
        group_id   INTEGER PRIMARY KEY,
        name       TEXT NOT NULL,
        parent_id  INTEGER REFERENCES student_group(group_id)  -- NULL = root
      );

New thing shown: parent_id is a foreign key pointing back into the SAME table.
Read it as a tree:
      All Students (1)
        |-- Grade 10 (2)
        |     |-- Grade 10 - Section A (4)
        |     +-- Grade 10 - Section B (5)
        +-- Grade 11 (3)
One self-pointing column stores a tree of ANY depth with no extra tables, and
the single NULL marks the one row with no parent. student-panel uses exactly
```

this shape twice (its groups tree and its parameters tree). It is confusion-free once you have seen the rows: a pointer that happens to aim at its own table.

>> PITFALL: beginners think SQL is "the hard part" and design is "the easy part." It is the reverse. CREATE TABLE is a near-mechanical translation of a good paper design. If the design is right, the SQL is short and obvious. All the hard thinking happens on paper, before you type anything. That is WHY the primer keeps you on paper so long. Look back at the five triples: in every one, the SQL (c) just re-states the paper line (a); the thinking was already done.

TRY IT (paper): pick any table from the samples PDF, cover its grid, and write its (a) paper attribute-list line yourself. Then uncover the grid and check you named the same columns. If you can go (a) -> (b) in your head, you can read any table in your materials.

BEFORE WE LEAVE PART 2 - THREE SHORT BRIDGE NOTES

The five triples above showed you the WHAT. Here are three names the primer leans on, dropped onto things you can already picture, so its words land.

(A) THE THREE HEIGHTS - AND THE MODEL YOU CHOSE

A database is not the world; it is a small, deliberate MODEL of the slice of the world you care about. The library model "knows" a book has a title and an isbn and nothing of its coffee stains. Design is choosing what to keep and letting the rest go. You make that choice at three HEIGHTS, highest first:

height	what you decide	the triple costume above
conceptual	the ideas: the things and how they connect, plain words	(a) the paper line student = (...)
logical	tables, columns, keys, links	(b) the grid of rows
physical	the real engine: types, indexes, where bytes sit	(c) the CREATE TABLE, and where the bytes live

Beginners dive to the physical (typing commands); good designers linger at the conceptual (getting the ideas right first). This whole companion lives at the top two heights; the practice lesson is where you finally touch the third.

(B) THE PAPER PIECES, NAMED PRECISELY (nouns, adjectives, verbs)

The three primitives map onto parts of speech - which is why good ER modelling sounds like plain talking:

- ENTITIES are the NOUNS (student, book, copy) -> become TABLES
- ATTRIBUTES are the ADJECTIVES (its name, its date) -> become COLUMNS
- RELATIONSHIPS are the VERBS (borrows, belongs to) -> become LINKS/FKs

Attributes come in four kinds, and each forces a different move. Three you have already met: ATOMIC (one plain column); MULTI-VALUED (its own table - confusion 7); DERIVED (do not store it - confusion 6). The fourth is COMPOSITE: one name hiding several values. SPLIT it into the parts you will search or sort by:

LUMPED (cannot sort by city, cannot greet by first name):

memb_id	full_name	address
M-1001	Sara Ahmadi	14 Rose Lane, Brighton

```
  SPLIT (each searchable part is its own column):
  +---------+------------+--------+--------------+----------+
  | memb_id | given_name | family | street       | city     |
  +---------+------------+--------+--------------+----------+
  | M-1001  | Sara       | Ahmadi | 14 Rose Lane | Brighton |
  +---------+------------+--------+--------------+----------+
```

And every relationship (verb) has a DEGREE - how many kinds of thing it joins:
  - UNARY : one table points at itself. Triple 5's student_group tree.
  - BINARY : two tables - the everyday case. Every link in Triples 2-4.
  - TERNARY: three at once. Confusion 8's doctor + drug + patient.
Naming the degree is just counting the boxes the verb reaches.

(C) A LAST WORD ON KEYS (candidate, composite, and a bridge's own id)

CANDIDATE KEY: often several columns could each identify a row - each unique and never empty. In Triple 2 both book_id and isbn qualify; each is a CANDIDATE. You crown ONE as the PRIMARY key - it must be unique, never empty, and STABLE (never needing to change) - and you keep the rest under a UNIQUE rule so they still cannot duplicate. (That is exactly confusion 3's surrogate id + unique code.)

COMPOSITE KEY: a key may be MORE THAN ONE column. A bridge of pairs is keyed by the pair itself - (student_id, course_id) together identify the enrolment, and neither alone is unique. Normal and common; most bridge tables look like this:

```
  enrolment  (PK = the PAIR (student_id, course_id)):
  +------------+-----------+-------------+
  | student_id | course_id | enrolled_on |
  +------------+-----------+-------------+
  |     1      |    10     | 2026-09-01  |   student 1 appears twice - fine
  |     1      |    11     | 2026-09-01  |   course 10 appears twice - fine
  |     2      |    10     | 2026-09-02  |   but no PAIR ever repeats
  +------------+-----------+-------------+
```

A BRIDGE (also called a junction, link, or associative table) may instead be given its OWN surrogate id - tidy to point at from elsewhere. If you do, you MUST add a UNIQUE rule on the natural combination, or duplicate pairings creep back in. student-panel does exactly this: its scores bridge a student and a parameter for a period, carry their own id, and keep that combination unique so the same thing is never scored twice for the same period.

PART 3 - READING THE DIAGRAMS FLUENTLY (A CROW'S-FOOT CHEAT-SHEET)

The primer (Part 4) and the samples PDF both draw relationships in "crow's-foot" notation. Most beginners can follow the words but freeze at the little symbols on the connecting line. This part is a cheat-sheet you can keep open beside any diagram. It also explains one thing neither file spells out loudly: the symbols are written MIRRORED on the two ends of a line, which is why the same relationship can show "o<" at one end and ">o" at the other.

THE FOUR SYMBOLS. A line connects two tables. Near EACH end there are up to two marks. The mark CLOSEST to the table says the MAXIMUM (one, or many); the mark just before it says the MINIMUM (zero = optional, or one = mandatory).

```
    |    a single bar      -> "one"        (exactly one on this side)
    o    a small circle    -> "zero"       (this side is OPTIONAL; may be none)
    <    a crow's foot      -> "many"       (this side can be many)
    ||   bar-bar            -> "one and only one"      (min 1, max 1)
    |<   bar then foot      -> "one or many"           (min 1, max many)
```

```
    o<   circle then foot   -> "zero or many"          (min 0, max many)
    o|   circle then bar     -> "zero or one"           (min 0, max 1)
```

KEY IDEA: read the symbols from the OUTSIDE in toward the table: the far mark is the minimum (o = optional, | = mandatory), the near mark is the maximum (| = one, crow's foot = many).

THE COMPLETE REFERENCE TABLE. Every combination you will meet, with its right-end form, its mirrored left-end form, the (min,max) it encodes, and the plain-English reading. "Right-end form" is how it looks when the table sits to the RIGHT of the line; "left-end form" is the mirror, table on the LEFT.

right-end	left-end	min	max	reads as
||	||	1	1	"one and only one" (exactly one)
o|	|o	0	1	"zero or one" (optional, at most one)
|<	>|	1	many	"one or many" (at least one)
o<	>o	0	many	"zero or many" (optional, many)

Notice "||" mirrored is still "||" - it is symmetric. The other three flip: the circle and the foot swap to the far side when the table moves to the left.

A TINY TWO-BOX EXAMPLE FOR EACH (read the mark nearest the RIGHT box):

one and only one (||):

```
    Loan  >o----------||  Copy
```

"each loan is of exactly one copy."

zero or one (o|):

```
    Employee  ||-------o|  ParkingSpot
```

"an employee has zero or one parking spot (maybe none, never two)."

one or many (|<):

```
    Order  ||----------|<  OrderLine
```

"an order has one or many lines - never zero (an empty order is silly)."

zero or many (o<):

```
    Member  ||---------o<  Loan
```

"a member has zero or many loans (a brand-new member has none yet)."

THE MIRRORING, SHOWN. Here is the SAME fact - "a member has zero-or-many loans; each loan has exactly one member" - drawn both ways round. The meaning does not change; only which side the circle-and-foot face:

```
member on the LEFT:          Member  ||--------o<  Loan
member on the RIGHT:         Loan   >o--------||   Member
```

In the first, "o<" sits by Loan. In the second, its mirror ">o" sits by Loan. Same loan end, same "zero or many," just facing the line from the other side. Do not let ">o" and "o<" look like two different ideas - they are one idea seen from the two ends.

WORKED READING 1 - the primer's library line (keep this one; it is the model):

```
    Member  ||-----o<  Loan  >o-----||  Copy
```

Read it in two halves, each outward-to-inward toward its table:

- Member side: "||" next to Member = one-and-only-one Member.
  "o<" next to Loan = zero-or-many Loans.

=> "one member has zero or many loans; each loan belongs to exactly one member."

```
  - Copy side:   ">o" next to Loan = zero-or-many Loans (mirrored o<).
                 "||" next to Copy = one-and-only-one Copy.
    => "one copy has zero or many loans; each loan is of exactly one copy."

Put together: a loan links exactly one member to exactly one copy; a member or
a copy may have no loans yet (the "o" = optional side), which is why a brand-
new member and a brand-new copy both still fit the diagram.

WORKED READING 2 - the simplest one-to-many:

    Publisher  ||-----o<  Book

  - "||" next to Publisher = one-and-only-one publisher.
  - "o<" next to Book       = zero-or-many books.
  => "one publisher has zero or many books; each book has exactly one
     publisher." The crow's foot is by BOOK, so BOOK carries publisher_id - the
     foot sits where the foreign key lives.

WORKED READING 3 - a many-to-many, and why it cannot stay a single line:

    Student  >o---------o<  Course

  - ">o" next to Student = zero-or-many students (mirrored o<).
  - "o<" next to Course   = zero-or-many courses.
  => "a student takes zero or many courses; a course has zero or many
     students." Both ends are crow's feet - a many-to-many. Neither side can
     hold the pointer (one cell cannot hold many ids), so this line MUST become
     a middle table. Resolved, it reads:

    Student  ||-----o<  Enrolment  >o-----||  Course

  => "a student has zero-or-many enrolments; each enrolment is for exactly one
     student; each enrolment is for exactly one course; a course has zero-or-
     many enrolments." The impossible single line became two clean one-to-
     manys through Enrolment. (That is confusion 8.)

WORKED READING 4 - a self-reference (a table pointing at itself):

     A self-reference is ONE box with a line that loops back to itself:

          +-----------------------------------------+
          |        "is the parent of"   (o<)        |
          v                                         |
     [ student_group:  group_id (PK) | name | parent_id (FK) ]
          |                                        ^
          |        "has as parent"      (o|)        |
          +-----------------------------------------+

Read the loop aloud: "a group is the parent of zero-or-many child groups; each
group has zero-or-one parent." The root ('All Students') has parent_id = NULL -
no parent. Part 2's Triple 5 shows this exact tree as rows. One box, one line
looping back: that is how a tree is drawn.

WORKED READING 5 - the full library chain from the primer's worked example:

    Member  ||--o<  Loan  >o--||  Copy  >o--||  Book

Read it as one sentence per segment:
  - "a member has zero or many loans; each loan has exactly one member."
  - "a copy has zero or many loans; each loan is of exactly one copy."
  - "a book has zero or many copies; each copy is of exactly one book."
```

Every sentence is true of a real library, so the chain is sound. If any sentence were false (say a copy could belong to two books), the diagram would be wrong - and saying it aloud is how you catch that before building.

TRY IT (paper): take any diagram in the samples PDF, cover the prose, and read the symbols aloud as full sentences the way we just did. If your sentence matches the PDF's caption, you can read crow's-foot. If not, bring the diagram to me and we will read it together.

>> PITFALL: forgetting that the crow's foot (many) sits on the end that holds the FOREIGN KEY. "Many loans per member" means the LOAN table carries member_id, not the other way around. The foot points at the table that does the pointing. We cover this again in Part 4, number 4.

PART 4 - THE CONFUSIONS ALMOST EVERY BEGINNER HITS

For each one: the confusion in plain words, the fix, WHERE it lives in your other materials, and HOW student-panel handles it (so you are not surprised later). Where the practice lesson or your own future design will differ from student-panel, I say so - that is a rule we keep. New in this edition: each confusion is also DRAWN, so the idea is a picture and not only a paragraph.

1) TYPE vs INSTANCE (table vs row)
   Confusion: "Is 'student' one thing or many things?"
   Fix: "student" the TABLE is the KIND; each ROW is one real student. The table is the cookie-cutter; the rows are the cookies. You design the cutter, not each cookie.
   Where: primer Part 2.1; samples PDF Lessons 1-2.
   student-panel: a "students" table (the cutter); every enrolled student is a row (a cookie).

   PICTURE - one cutter, many cookies:

     THE CUTTER (the table design = the TYPE; the shape, no people yet)

student_id	full_name	joined_on

     THE COOKIES (the ROWS = instances; each one is a real student)

1	Sara Ahmadi	2026-09-01
2	Omid Karimi	2026-09-03
3	Lena Hassan	2026-09-05

   You design the header once; the rows are stamped out from it. "student" (singular, the design) is one thing; the students (the rows) are many.

2) ENTITY vs ATTRIBUTE (its own table, or just a column?)
   Confusion: "Should 'group' be a column on student, or its own table?"
   Fix: ask "do I keep facts ABOUT this thing, or will many rows share it?" If yes to either, it is an ENTITY (own table) and you point at it with a foreign key. If it is just one plain value that belongs to this one row, it is an ATTRIBUTE (a column). "Colour" is a column in a small shop and a table in a paint factory - purpose decides.
   Where: primer Part 2.1-2.2; samples PDF Lesson 2 (the colour example).
   student-panel: "group" is its own table (groups have names, parents, and members), so a student's group is a foreign key, not a text column.

PICTURE - the same "group" done both ways:

AS A PLAIN COLUMN (ok only if a group is JUST a label, never shared):

student_id	full_name	group	
1	Sara Ahmadi	Grade 10 A	
2	Omid Karimi	Grade 10 A	<- retyped; rename it once and these two now disagree

AS ITS OWN TABLE (needed once a group has facts, or is shared by many):

student_group:

group_id	name
7	Grade 10 A

student (its group_id POINTS at student_group.group_id):

student_id	full_name	group_id
1	Sara Ahmadi	7
2	Omid Karimi	7

The name "Grade 10 A" now lives in ONE row; rename it there and every student follows - no retyping, no disagreement.

3) NATURAL vs SURROGATE KEY - and INT vs UUID

Confusion: "What do I put in the id column, and why not just use the name or the email?"

Fix: a NATURAL key is real-world (email, ISBN, national id). A SURROGATE key is invented by the system purely to be a stable handle. Natural keys change and repeat; surrogate keys never do. Modern practice: give each table a surrogate primary key AND keep the useful natural value under a UNIQUE rule so it still cannot duplicate.

Two kinds of surrogate exist, and here is a real difference between your materials:

- the samples PDF uses INTEGER auto-numbers (1, 2, 3 ...) everywhere. Simple, and great for learning.
- the primer (Part 3) and student-panel use a UUID - a long random id like 7f3a...-...-9c2b - PLUS a human-friendly unique code.

Both are surrogate keys; neither is wrong. INT is easier to read while learning; UUID is better when ids must be unique across machines or must not reveal "how many" or "what order." This is a DIFFERENCE you will meet, not a contradiction.

Where: primer Part 3 (it even notes student-panel's UUID + student_code); samples PDF Lessons 3-4 (INT ids).

student-panel: UUID primary keys, plus a unique human student_code. So when you see "1, 2, 3" in the PDF and long random ids in student-panel, you are seeing the same idea in two costumes. In the PRACTICE lesson we will use INT ids (like the PDF) because they are easier to type and read - a place where the practice deliberately differs from student-panel.

PICTURE - the SAME table, INT id vs UUID + human code:

INT surrogate (samples PDF / practice style):

student_id	full_name
1	Sara Ahmadi
2	Omid Karimi

```
+------------+--------------+
```

UUID surrogate + a unique human code (primer / student-panel style):

```
+--------------------------------------+--------------+-------------+
| student_id (UUID)                    | student_code | full_name   |
+--------------------------------------+--------------+-------------+
| 7f3a1c6e-9b2d-4a51-8e07-2c9b1f4d6a30 | STU-0001     | Sara Ahmadi |
| b1d9e4a2-3c77-4f16-9a5e-6d8c0b2a1e55 | STU-0002     | Omid Karimi |
+--------------------------------------+--------------+-------------+
```

BOTH student_id columns are surrogate keys (invented, meaningless, stable). student_code is the NATURAL value kept UNIQUE for humans to search by. The INT and the UUID are the same idea - a stable handle - in two costumes.

4) WHERE THE FOREIGN KEY GOES (which table carries the pointer?)
Confusion: "Does the loan point at the member, or the member at the loan?"
Fix: in a one-to-many, the foreign key lives on the MANY side. One member, many loans -> loan carries member_id. Rule of thumb: the child points at the parent; the crow's foot sits where the FK lives.
Where: primer Part 3 (FK placement); samples PDF Lessons 7-8.
student-panel: a score/evaluation row points at the student and the parameter it is about; the "many" rows carry the pointers, exactly as the rule says.

PICTURE - right vs wrong placement:

RIGHT - FK on the MANY side (each loan carries one member_id):
member (the "one" side):

```
+-----------+-------+
| member_id | name  |
+-----------+-------+
|     1     | Sara  |
|     2     | Omid  |
+-----------+-------+
```

loan (the "many" side - member_id is the pointer):

```
+---------+-----------+-------------+
| loan_id | member_id | borrowed_on |
+---------+-----------+-------------+
|    1    |     1     | 2026-09-10  |
|    2    |     1     | 2026-09-02  |
|    3    |     2     | 2026-09-13  |
+---------+-----------+-------------+
```

follow each loan's member_id back to member: loans 1 & 2 -> Sara (1), loan 3 -> Omid (2). Many loans point at one member; one value per cell.

WRONG - FK on the ONE side (member tries to list its loans):

```
+-----------+-------+-----------+
| member_id | name  | loan_ids  |
+-----------+-------+-----------+
|     1     | Sara  | 1, 2      |  <- a LIST in a cell: breaks 1NF, cannot
|     2     | Omid  | 3         |     be searched or kept honest
+-----------+-------+-----------+
```

The crow's foot (many) sits on LOAN, so the pointer sits on LOAN.

5) WHAT "NULL" MEANS (empty on purpose, or missing by accident?)
Confusion: "A blank cell - is that allowed? what does it mean?"
Fix: NULL means "no value here." You must DECIDE, per column, whether blank is legal and what it means. loan.returned_on being NULL means "not returned yet" - a real, useful meaning. A NULL email might mean "we never collected it." Give every nullable column a sentence of meaning; forbid NULL (NOT

NULL) where blank would be nonsense.
Where: primer Part 2.2 and Part 2.4 (participation = the optional/mandatory choice); samples PDF Lesson 8 and the delete-rule tables.
student-panel: it uses NULL deliberately - e.g. an assignment can point at EITHER a group OR a single student, so one of those two pointers is NULL at a time (the "either/or" pattern). We will look closely at that when we reach it.

PICTURE - two honest, meaningful NULLs:

(i) returned_on NULL = "still out" (a useful fact, not missing data):

loan_id	member_id	borrowed_on	returned_on	
1	1	2026-09-10	(NULL)	<- still out
2	1	2026-09-02	2026-09-09	<- returned

(ii) the EITHER/OR pattern (an assignment aims at a group OR one student):

assignment_id	group_id	student_id	
1	7	(NULL)	<- aimed at a whole group
2	(NULL)	3	<- aimed at one single student

Exactly one of group_id / student_id is filled; the other is NULL ON PURPOSE. The blank is doing real work: it says "not this kind of target."

6) DERIVE, DON'T STORE - and the one big exception
Confusion: "Should I keep a 'total' column, or an 'age' column?"
Fix: if a value can be computed from other values, usually do NOT store it; compute it when asked. Store birth_date, show age. Store line items, add up the total. Storing a derived value means it can drift out of step with the thing it came from.
THE EXCEPTION (important, and this is where the samples PDF ADDS something the primer only hints at): a value you capture ON PURPOSE at a moment in time, because it must never change afterward, is NOT a derived value - it is a FROZEN FACT. The classic case is unit_price_at_sale on an order line. The product's price today is irrelevant; the order must remember what the customer actually paid. Keep it. It only LOOKS like a redundancy.
Where: primer Part 2.2 and Part 6 (derive-don't-store, denormalise); samples PDF Lessons 9 and 16 ("Not a break: unit_price_at_sale is a frozen fact") - a genuinely good addition worth folding into your notes.
student-panel: a recorded score is a frozen fact of its period - once a period is scored, that number stands even if the parameter's setup later changes. Same principle as unit_price_at_sale.

PICTURE - don't store what you can compute; DO store what was true once:

DON'T store 'age' (it is stale the next birthday; birth_date never is):

student_id	birth_date	age	
1	2008-05-01	18	<- WRONG: wrong the day after a b-day

keep birth_date; compute age = (today - birth_date) only when asked.

DO store a FROZEN FACT (the price actually paid, captured on purpose):
product (its price TODAY):

product_id	price

```
+------------+-------+
|    100     | 12.00 |
+------------+-------+
order_line (the price captured AT THE SALE):
+---------+------------+-----+--------------------+
| line_id | product_id | qty | unit_price_at_sale |
+---------+------------+-----+--------------------+
|    1    |    100     |  2  |       10.00        |
+---------+------------+-----+--------------------+
product.price is 12.00 now; the sale remembers 10.00 - what was really
paid. That is not drift; it is history captured on purpose. In student-
panel a scored period is this same frozen fact: the number stands.
```

7) ONE VALUE PER CELL (first normal form, in plain words)
   Confusion: "Can I just put 'math, science, art' in one subjects column?"
   Fix: no. One cell, one value. A list in a cell cannot be searched, counted, or constrained properly ("who takes science?" becomes guesswork). Repeating columns (phone1, phone2, phone3) are the same mistake wearing a different hat. The fix is always: give the repeating thing its OWN table with a foreign key back.
   Where: primer Part 6 (1NF); samples PDF Lesson 14 (phone1/2/3 and the list-in-a-cell break). You will FEEL this one in the practice lesson by trying to query a comma-list and failing.
   student-panel: multi-valued things (a student's many scores, a group's many members) live in their own tables, never as lists in a cell.

   PICTURE - the comma-list smell, and the child-table cure:

```
  BAD - three values crammed into one cell (cannot be searched or counted):
  +------------+-------------+---------------------+
  | student_id | full_name   | subjects            |
  +------------+-------------+---------------------+
  |     1      | Sara Ahmadi | math, science, art  |  <- 3 values, 1 cell
  |     2      | Omid Karimi | math                |
  +------------+-------------+---------------------+
  "who takes science?" -> you must pattern-match inside text and hope.

  ALSO BAD - the same mistake as repeating columns:
  +------------+-------------+----------+----------+----------+
  | student_id | full_name   | subject1 | subject2 | subject3 |
  +------------+-------------+----------+----------+----------+
  |     1      | Sara Ahmadi | math     | science  | art      |  <- a 4th?
  +------------+-------------+----------+----------+----------+

  FIXED - the repeating thing gets its OWN table, one value per row:
  student:                     student_subject (one row per pair):
  +------------+----------+    +------------+-----------+
  | student_id | name     |    | student_id | subject   |
  +------------+----------+    +------------+-----------+
  |     1      | Sara     |    |     1      | math      |
  |     2      | Omid     |    |     1      | science   |
  +------------+----------+    |     1      | art       |
                               |     2      | math      |
                               +------------+-----------+
  now "who takes science?" is one clean lookup: rows where subject='science'.
```

8) THE ONE MOVE: when to add a table "in the middle"
   Confusion: "This relationship feels messy - what do I do?"
   Fix: three different-looking problems all have the SAME solution - put a new table between the two things:
      - MANY-TO-MANY (students <-> courses): a bridge table (enrolment).

- A LINK THAT HAS ITS OWN FACTS (the enrolment has a grade and a date):
  the same bridge, now carrying those facts.
- A THREE-WAY fact (doctor + drug + patient = a prescription): a table
  in the middle holding all three pointers.

If a relationship has its own attributes, or joins more than two things, or is many-to-many, it wants to BE a table, not a line.
Where: primer Part 5 (the five hard patterns); samples PDF Lessons 10-13 ("three triggers, one idea").
student-panel: its scores/evaluations table is exactly this - a table in the middle that links a student and a parameter for a period and carries the score itself. The thing that felt like "just a link" is actually the most important table in the design.

PICTURE - three messy shapes, one cure (a table in the middle):

(a) MANY-TO-MANY -> a bridge table of pairs:

```
Student  >o-----------o<  Course     (cannot be stored as one line)
becomes
Student  ||--o<  Enrolment  >o--||  Course
```

enrolment (one row per student-course pair):

student_id	course_id
1	10
1	11
2	10

(b) A LINK WITH ITS OWN FACTS -> the SAME bridge, carrying the facts:

student_id	course_id	grade	enrolled_on
1	10	A	2026-09-01
1	11	B	2026-09-01

grade and enrolled_on belong to the PAIR, so they live on the bridge.

(c) A THREE-WAY FACT -> a middle table holding three pointers:

```
Doctor  ----\
Drug    -----)--- "prescribes" --->  prescription
Patient ----/
```

prescription_id	doctor_id	drug_id	patient_id
1	20	30	40

one row ties a doctor, a drug, and a patient into a single fact.

All three: the "link" became a TABLE. student-panel's scores table is cases (b)+(c) at once - a middle table linking student x parameter x period that carries the score.

9) RESERVED WORDS (why you cannot name a table "order" or "group")
Confusion: none at first - then a baffling error when a table named "order" or "group" or "user" refuses to work.
Fix: those words already mean something to the database (ORDER BY, GROUP BY, USER). Avoid them as plain names: use "orders", "groups", "app_user", or quote them. This is a PHYSICAL-level gotcha the primer does not mention and the samples PDF flags - a useful addition.
Where: samples PDF "How to read" note on reserved words.

student-panel: runs on Postgres, where "user", "order", and "group" are all reserved, so its real tables use safe names. Worth remembering before you name anything in the practice lesson.

PICTURE - names that fight the database, and their safe replacements:

the word you WANT	why it fights	the SAFE name to use
order	means ORDER BY ...	orders
group	means GROUP BY ...	groups
user	means CURRENT_USER	app_user
select / table / ...	all SQL keywords	pick a non-keyword noun

```
CREATE TABLE order  ( ... );   -> may ERROR, or force you to quote "order"
CREATE TABLE orders ( ... );   -> safe, boring, works everywhere
```

Boring names are a feature. Postgres (student-panel's engine) treats all three above as reserved, which is exactly why student-panel never names a table plainly "user" or "group".

10) SOFT DELETE vs HARD DELETE (does "delete" really erase?)
Confusion: "If I remove a student, is the data gone forever?"
Fix: a HARD delete removes the row for good. A SOFT delete keeps the row but marks it (e.g. an archived_at date or an is_active flag) so it stops showing up while the history it is attached to survives. Which to use is a design decision about whether you ever need the past back.
Where: primer Part 9 (design judgement, soft vs hard delete); the samples PDF shows HARD deletes with ON DELETE rules (RESTRICT/CASCADE/SET NULL).
student-panel: SOFT-deletes / archives rather than erasing, so past scores stay meaningful. In the PRACTICE lesson we will use HARD deletes on a throwaway database - that is simpler and lets you SEE referential integrity refuse an orphan. So the practice deliberately differs from student-panel here: practice erases, student-panel archives. Keep that difference in mind.

PICTURE - the same "remove Omid," two ways:

BEFORE (both rows present; archived_at empty = active):

student_id	full_name	archived_at
1	Sara Ahmadi	(NULL)
2	Omid Karimi	(NULL)

HARD DELETE (practice, throwaway DB) - the row is simply GONE:

student_id	full_name	archived_at
1	Sara Ahmadi	(NULL)

Omid no longer exists; any history that pointed at him is now orphaned (which is why the database may refuse the delete - referential integrity).

SOFT DELETE (student-panel) - the row STAYS, just marked and hidden:

student_id	full_name	archived_at
1	Sara Ahmadi	(NULL)
2	Omid Karimi	2026-10-02

<- hidden from lists; past kept

Omid drops out of normal views, but his old scores still make sense

because the row he points from is still there. That is why student-panel archives instead of erasing.

THOSE ARE THE TEN YOU WILL MEET FIRST. A FEW MORE, ALMOST AS COMMON:

The ten above are the classics. These next few trip up newcomers nearly as often, and each is a pattern the primer teaches - same shape as before: confusion, fix, WHERE, student-panel, and a picture.

11) ONE-TO-ONE: MERGE IT, OR SPLIT IT ON PURPOSE
Confusion: "Two things line up exactly one-to-one - must I use two tables?"
Fix: usually NO. A true 1:1 is a hint the two are really ONE thing, so keep the extra columns on the main table (they sit NULL when absent). You SPLIT on purpose only for a reason you can name: the extra data is rare, or sensitive, or bulky. When you split, the foreign key goes on the NEVER-EMPTY side and wears UNIQUE + NOT NULL - those two constraints are what hold it at 1:1 and stop it sliding into 1:many.
Where: primer Part 5 (one-to-one) and Part 2 (optional vs mandatory sides). student-panel: keeps rarely-present or sensitive facts in their own place by the same judgement - not a rule, a reason.

PICTURE - the same person-and-passport, merged vs split:

MERGED (the usual answer; passport columns are NULL when there is none):

person_id	name	passport_no	passport_expiry	
1	Sara Ahmadi	981222345	2030-05-01	
2	Omid Karimi	(NULL)	(NULL)	<- none

SPLIT ON PURPOSE - two tables, the link carried on ONE side:
person (the never-empty side):

person_id	name
1	Sara Ahmadi
2	Omid Karimi

passport (FK person_id is UNIQUE so one person cannot own two rows, and NOT NULL so a passport row cannot float free):

person_id	number
1	981222345

Person 2 simply has no row here - that is the "zero" in "zero or one."

12) WEAK ENTITIES - AND WHAT "DELETE THE PARENT" DOES TO THE CHILDREN
Confusion: "An order-line, a reply, a room - does it get its own table?"
Fix: yes, but it is a WEAK entity - it cannot exist without its parent. An order-line is meaningless without its order; "line 2" only means "line 2 OF order 88." So its foreign key to the parent is MANDATORY (NOT NULL) and part of its very identity: its key is the parent's id PLUS a local number - a COMPOSITE key (order_id, line_no). A weak entity never floats free.
Where: primer Part 5 (weak entities) and Part 3 (the ON DELETE rules). student-panel: a score is weak - it belongs to a student and a period and cannot stand alone; because student-panel SOFT-deletes, parents stay, so children are never orphaned (confusion 10).

PICTURE - an order and its lines (table is "orders", not "order" - see #9):

```
  orders (strong):             order_line (weak; PK = (order_id, line_no)):
  +----------+------------+    +----------+---------+----------+-----+
  | order_id | placed_on  |    | order_id | line_no | product  | qty |
  +----------+------------+    +----------+---------+----------+-----+
  |    88    | 2026-09-30 |    |    88    |    1    | Pencils  |  3  |
  |    89    | 2026-10-01 |    |    88    |    2    | Eraser   |  1  |
  +----------+------------+    |    89    |    1    | Notebook |  2  |
                               +----------+---------+----------+-----+
```

line_no alone means nothing; order_id is mandatory; a line with no order cannot exist.

When the PARENT is deleted the database needs ONE plain answer. Say each one aloud and keep the one that is TRUE of your world:

- CASCADE : "delete the order and its lines go with it" - they cannot outlive it; the usual choice for a weak child.
- RESTRICT: "you may NOT delete the order while any line still points at it" - the safe default for strong parents.
- SET NULL: "keep the child, just empty its pointer" - only possible when the link is OPTIONAL, so it is wrong for a weak child (whose FK is NOT NULL).

13) WHY SPLIT INTO SO MANY TABLES? THE THREE ANOMALIES (and 2NF, 3NF)
Confusion: "One wide table is simpler - why break it into pieces?"
Fix: because repetition rots. Cram facts about two different things into one table and the SAME fact gets written many times - then the copies drift apart. One value per cell was confusion 7 (that was 1NF); this is the rest of the story. The cure is to split so each fact lives in exactly ONE place.
Where: primer Part 1 and Part 6 (normalisation and the three anomalies).
student-panel: facts about a student live on the student, facts about a score on the score - never copied side by side, so they cannot disagree.

PICTURE - one wide loan sheet, and the three ways it goes wrong:

```
  +---------+-------------+--------------+------------+
  | loan_id | member_name | member_phone | book_title |
  +---------+-------------+--------------+------------+
  |    1    | Sara Ahmadi | 555-1020     | Dune       |
  |    2    | Sara Ahmadi | 555-1020     | Emma       |  <- phone written
  |    3    | Omid Karimi | 555-7788     | Dune       |     twice already
  +---------+-------------+--------------+------------+
```

- UPDATE anomaly: Sara changes phone; you fix row 1, miss row 2 -> the table now claims two phones and cannot say which is right.
- INSERT anomaly: a new member who has borrowed nothing has NO row to sit in (the sheet is about loans) -> you cannot record her at all.
- DELETE anomaly: delete Omid's only loan (row 3) and Omid himself vanishes -> you meant to drop a loan and dropped a person.

CURE: split into member / book / loan; loan points at the other two by id. Sara's phone now sits in ONE place; a loan-less member is fine; deleting a loan loses only the loan.

Two deeper splits, each a one-line rule, each fixed by "move the fact home":

- 2NF: on a table keyed by a PAIR (student_id, course_id), the student's NAME depends on student_id ALONE - only PART of the key - so it does not belong here; it belongs on the student table.
- 3NF: a book row holding publisher_city is wrong - the city depends on the publisher (another NON-key column), not on the book; move it onto a publisher table.

The whole craft fits one sentence: every non-key fact must depend on "the key, the whole key, and nothing but the key." (You MAY, knowingly and for measured speed, keep a little redundancy - DENORMALISE - but correct first,

fast later, and never by accident.)

14) A FLEXIBLE (JSON) COLUMN vs PROPER COLUMNS
Confusion: "Can I just dump the extra, varies-per-row fields into one column?"
Fix: most engines let a single column hold a free-form bag of data (often JSON). It is tempting, but the database cannot easily check, key, sort, or search INSIDE that bag. Rule of thumb: any fact you will FILTER, SORT, or ENFORCE deserves a real column of its own; only genuinely open-ended extras belong in a flexible column. Reach for it rarely, and on purpose.
Where: primer Part 9 (a flexible column vs proper columns).
student-panel: the facts it reports on and filters by are real columns; it does not hide a queryable fact inside a blob.

PICTURE - which facts earn a real column:

the fact	real column?	why
a student's birth_date	REAL column	you sort and filter by it
a score's value	REAL column	you total it and chart it
which group a student is in	REAL column	you filter by it; it is a link
a free-form per-import note	JSON is fine	open-ended, never queried

---

PART 5 - DO THE OTHER AI PIECES AGREE WITH THE PRIMER?

Short answer: yes, almost entirely - they were written to sit alongside the primer and even tell you to compare notes. They use the same examples and the same crow's-foot notation, and on every big idea (keys, foreign keys, the five patterns, the three normal forms) they say the same thing in a different voice. In a few places they ADD something useful the primer underplays (the frozen-fact exception, reserved words, the explicit mirroring of crow's-foot symbols, a surrogate-id option for bridge tables). The one thing to ADD is that the samples PDF uses only INTEGER ids and never mentions UUID - which is exactly what student-panel uses - so a learner following only the PDF would be surprised by student-panel's long random ids (see Part 4, number 3).

THE WHOLE VERDICT AT A GLANCE:

what the pieces do	verdict
1NF / 2NF / 3NF and the three anomalies	AGREE (identical to the primer)
"the key, the whole key, nothing but..."	AGREE (same mnemonic)
foreign key on the MANY side	AGREE
1:1 placement (on the never-empty side)	AGREE
self-reference tree, NULL at the root	AGREE
the "one move" (M:N / facts / ternary)	AGREE (same as primer Part 5)
ON DELETE RESTRICT / CASCADE / SET NULL	AGREE
frozen-fact exception (unit_price)	ADD - useful; primer only hinted
reserved words (order/group/user)	ADD - useful physical gotcha
crow's-foot mirroring spelled out	ADD - folded into Part 3 here
a surrogate-id option for a bridge	ADD - a valid alternative to show
INTEGER ids only; UUID never mentioned	THE ONE GAP (student-panel uses UUID)

So: a strong team, with four nice additions and exactly one thing to add. There are no outright contradictions in the readable material - only choices of emphasis. (One item is still pending: the lessons .docx has ten practice questions whose written answers have not yet been checked line by line; that is flagged in the review file below and is the one thing left to certify.)

The full, point-by-point comparison - what agrees, what differs, what needs

adding, and which side is right and why - is written out separately in:

    database-design-AI-pieces-review.txt

Read that when you want the detailed verdict. This companion only needed to reassure you that your materials are a team, not a tug-of-war.

PART 6 - WHAT TO DO WITH THIS COMPANION

---

1. Read it once, slowly, with the primer nearby. When Part 4 points at a primer part, flip there and reread that bit in light of the confusion we just named.
2. Keep the Part 3 cheat-sheet handy whenever you look at a diagram.
3. THEN open the practice lesson (database-design-practice-lesson.txt) and do it with the in-browser sandbox open. Reading made these ideas concrete; doing will make them yours.
4. Write down any question that does not resolve. Bring the list to me. We do NOT move on to re-deriving student-panel until your questions are answered - there is no rush, and confusion now is cheaper than confusion later.

A READINESS CHECK before the practice lesson. You do not need full marks - this just shows you where to reread. Can you, from memory:

- [ ] say the paper word for a table, a row, and a column (Part 2)?
- [ ] turn a one-line paper design into a CREATE TABLE in your head (Part 2)?
- [ ] read Member ||--o< Loan aloud as two true sentences (Part 3)?
- [ ] say which side of a one-to-many carries the foreign key (Part 4 #4)?
- [ ] name the "one move" that fixes M:N, a link-with-facts, and a ternary (Part 4 #8)?
- [ ] explain why 'math, science, art' in one cell is a problem (Part 4 #7)?
- [ ] state one honest meaning for a NULL (Part 4 #5)?
- [ ] say how student-panel's ids and deletes differ from the practice lesson's (Part 4 #3 and #10)?

Any box you cannot tick is just a signpost back to that Part - no penalty. Tick most of them and the practice lesson will feel like recognition, not surprise.

KEY IDEA for the whole file: design lives on paper; a database is just that paper design, made real and filled with rows. Everything you are learning is how to make the paper good BEFORE it becomes real.

A reminder of our standing rule: we never change the student-panel project. You will build your own version in student-panel-I, and whenever your way differs from student-panel's way, I will point it out - as I already did in Part 4 for ids (INT vs UUID) and deletes (hard vs soft).

- end of companion -